GestoLivre — Biblioteca de Gestos por Webcam com Treino Personalizado

Biblioteca open source + app de treino para ensinar a webcam a reconhecer os *seus* movimentos e disparar ações, com foco em acessibilidade.

Versão 0.1 (proposta) · Execução 100% local no navegador · Metodologia: SDD + desenvolvimento incremental por MVP

1. Visão geral

1.1 A ideia em uma frase

Você abre a webcam, grava alguns exemplos de um movimento (ex.: inclinar a cabeça, piscar o olho esquerdo, abrir a boca, fazer um "joinha"), o sistema treina um modelo leve **em segundos, no próprio navegador**, e a partir daí esse movimento passa a executar uma ação (pressionar uma tecla, clicar, falar uma frase, chamar um webhook).

1.2 Por que isso é útil

- **Acessibilidade:** pessoas com mobilidade reduzida (lesão medular, ELA, paralisia cerebral, tetraplegia) podem controlar o computador com movimentos que *elas* conseguem fazer de forma confiável. Cada corpo é diferente, então o treino personalizado é o diferencial frente a gestos pré-definidos.
- **Projetos criativos e jogos:** controlar apresentações, jogos, instrumentos musicais ou automações domésticas por gestos.
- **Educação:** ensinar visão computacional e aprendizado de máquina de forma tangível (treinar um modelo em 30 segundos e ver funcionando).
- **Privacidade:** o vídeo nunca sai do dispositivo; só pontos de referência (landmarks) numéricos são usados e armazenados.

1.3 Produtos entregues

1. `@gestolivre/core` — biblioteca TypeScript (câmera, extração de landmarks, treino, inferência, eventos).
2. `@gestolivre/ui` — componentes React/Web Components prontos para a tela de treino (gravar, revisar, testar).
3. **GestoLivre Studio** — aplicação web (PWA) que usa as duas bibliotecas acima para o usuário final treinar gestos e mapear ações.

4. **GestoLivre Bridge** *(opcional, fase 2)* — pequeno app desktop que recebe eventos e injeta teclado/mouse no sistema operacional.

1.4 Personas

Persona	Necessidade
Usuária com mobilidade reduzida	Controlar teclado/mouse com movimentos de cabeça, rosto ou uma mão, calibrados para ela
Cuidador / terapeuta	Configurar e ajustar gestos para o paciente sem programar
Desenvolvedor / maker	Importar a biblioteca e reagir a gestos personalizados em seu app
Estudante / professor	Usar como laboratório de ML com resultados imediatos

2. Escopo

2.1 MVP (Fase 1)

- Captura da webcam no navegador com permissão explícita.
- Extração de landmarks de **mão**, **rosto (com blendshapes)** e **corpo (pose)**.
- Gravação de exemplos por gesto, com contagem regressiva e feedback visual.
- Treino local e incremental de um classificador leve, com avaliação imediata.
- Modo "teste ao vivo" mostrando o gesto detectado e a confiança.
- Mapeamento gesto → ação (teclas, clique, fala por voz sintetizada, URL/webhook).
- Salvar/carregar/exportar perfis de gestos (JSON + modelo) localmente.

2.2 Fase 2

- Gestos **dinâmicos** (movimentos ao longo do tempo: acenar, círculo com a mão, balançar a cabeça).
- Bridge desktop para controlar qualquer aplicativo do sistema.
- Modo **varredura** (scanning) com um único gesto como "botão" para teclado virtual.
- Controle de cursor por cabeça ou olhar aproximado, com *dwell click*.

2.3 Fora do escopo

- Reconhecimento facial para identificar pessoas.

- Envio de vídeo ou imagens para servidores.
- Rastreamento ocular de precisão clínica (exige hardware dedicado).
- Dispositivo médico certificado: o software é auxiliar e não substitui tecnologia assistiva clínica.

3. Metodologia de aprendizado de máquina

3.1 Decisão central: landmarks, não pixels

Em vez de treinar uma rede neural sobre imagens (caro, lento, exige milhares de exemplos), o sistema usa um modelo pré-treinado **apenas para extrair pontos de referência** do corpo/rosto/mão e treina um classificador pequeno sobre esses números. Vantagens:

- Poucos exemplos bastam (20 a 60 por gesto).
- Treino em segundos, no navegador, sem GPU dedicada.
- Robusto a iluminação, fundo e roupa.
- Privacidade: nenhuma imagem é armazenada.
- Transparência: é possível inspecionar e explicar o que o modelo vê.

3.2 Pipeline

```
Webcam → quadro de vídeo → detector de landmarks (MediaPipe)
      → pré-processamento (normalização) → vetor de características
      → classificador (personalizado) → suavização temporal
      → decisão (limiar + histerese + tempo de sustentação)
      → evento de gesto → executor de ação
```

3.3 Extração de características

Fonte	Saída do modelo	Uso
Mãos	21 pontos 3D por mão + lateralidade	Gestos de mão estáticos (joinha, paz, punho, apontar)
Rosto	468+ pontos + **52 blendshapes** (ex.: `jawOpen`, `eyeBlinkLeft`, `mouthSmileLeft`, `browInnerUp`)	Piscar, sorrir, abrir a boca, levantar sobrancelha, inclinação da cabeça
Corpo	33 pontos de pose	Gestos de braços e tronco

Normalização (obrigatória):

1. Translação: subtrair um ponto de referência (ex.: punho da mão, ponta do nariz).
2. Escala: dividir pela distância característica (ex.: largura da palma, distância entre os olhos).
3. Rotação opcional: alinhar eixo principal, quando o gesto não depender da orientação.
4. Espelhamento coerente (câmera frontal costuma ser espelhada).
5. Concatenar só as fontes habilitadas para o gesto (um gesto pode usar "mão + rosto").

3.4 Classificadores (do mais simples ao mais forte)

Nível	Técnica	Quando usar
1	**k-NN** sobre vetores normalizados (distância cosseno/euclidiana)	MVP: treino instantâneo, bom com poucos exemplos, explicável
2	**MLP pequena** (2 camadas densas, dropout) em TensorFlow.js	Mais exemplos e gestos parecidos entre si
3	**Janela temporal + 1D-CNN/GRU** ou **DTW** (Dynamic Time Warping)	Gestos dinâmicos (Fase 2)

A arquitetura expõe uma interface `Classifier` única; trocar a técnica não altera o resto do sistema.

3.5 Classe "nenhum gesto" (neutro)

O erro mais comum em sistemas assim é disparar ações sem querer. Por isso:

- O treino **sempre** inclui uma classe `neutro`, gravada com o usuário parado, falando, mexendo-se naturalmente e fora do quadro.
- O estúdio avisa se o gesto é muito parecido com o neutro (distância entre centróides abaixo do limiar).

3.6 Decisão robusta (anti-disparo acidental)

- **Limiar de confiança** por gesto (padrão 0,85, ajustável).
- **Suavização temporal:** média móvel das probabilidades em N quadros (padrão 5).
- **Tempo de sustentação (hold):** o gesto precisa durar T ms (padrão 300 ms; ajustável até vários segundos para quem tem tremor).
- **Histerese:** limiar de entrada mais alto que o de saída, evitando oscilar.
- **Cooldown:** intervalo mínimo entre disparos do mesmo gesto.
- **Modo repetição:** disparo único, contínuo (enquanto mantiver) ou pressionar/soltar.

3.7 Coleta guiada de dados

- Contagem regressiva e captura automática de 1 a 3 quadros por segundo até atingir a meta (ex.: 40 exemplos).
- Instrução para **variar** posição, distância, ângulo e iluminação durante a gravação (aumenta generalização).
- **Aumento de dados** leve sobre os landmarks: ruído gaussiano pequeno, pequenas rotações e escalas.
- Indicador de qualidade em tempo real: rosto/mão visível, boa iluminação, estabilidade da detecção.

3.7.1 Avaliação

- **Validação imediata:** separar 20% dos exemplos (por blocos temporais, não aleatório, para evitar vazamento entre quadros consecutivos) e mostrar acurácia e matriz de confusão simples.
- **Métricas de pesquisa/benchmark:** precisão, revocação e F1 por gesto; taxa de **disparos falsos por hora** em uso neutro; latência ponta a ponta.
- **Teste de estresse de falso positivo:** gravar 10 minutos de uso normal (digitando, conversando) e contar disparos indevidos.

4. Requisitos funcionais

Prioridade: **M** = Must, **S** = Should, **C** = Could.

4.1 Câmera e detecção (CAM)

ID	Requisito	Prior.
RF-CAM-01	Solicitar permissão da webcam de forma explícita e explicar o uso dos dados antes do pedido	M
RF-CAM-02	Listar e permitir escolher a câmera; lembrar a escolha	M
RF-CAM-03	Exibir pré-visualização espelhada com sobreposição dos landmarks (ativável)	M
RF-CAM-04	Processar quadros em Web Worker/OffscreenCanvas para não travar a interface	S

ID	Requisito	Prior.
RF-CAM-05	Alternar entre detectores (mão, rosto, corpo) para economizar processamento	M
RF-CAM-06	Mostrar aviso quando a detecção estiver ruim (pouca luz, fora do quadro)	S
RF-CAM-07	Parar a câmera imediatamente e mostrar indicador claro quando estiver ativa	M

4.2 Treino (TRN)

ID	Requisito	Prior.
RF-TRN-01	Criar, renomear, remover gestos e definir quais fontes usam (mão, rosto, corpo)	M
RF-TRN-02	Gravar exemplos com contagem regressiva, meta de quantidade e barra de progresso	M
RF-TRN-03	Revisar exemplos gravados (miniatura de landmarks) e excluir os ruins	M
RF-TRN-04	Treinar o modelo localmente com um clique, mostrando progresso e acurácia	M
RF-TRN-05	Treino incremental: adicionar exemplos a um gesto existente sem recomeçar	S
RF-TRN-06	Gravar automaticamente a classe `neutro` com assistente guiado	M
RF-TRN-07	Avisar quando dois gestos forem muito parecidos e sugerir regravar	S
RF-TRN-08	Escolher o classificador (k-NN ou MLP) com padrão recomendado	C

4.3 Reconhecimento e ações (ACT)

ID	Requisito	Prior.
RF-ACT-01	Modo "teste ao vivo" mostrando gesto atual, confiança e latência	M

ID	Requisito	Prior.
RF-ACT-02	Emitir eventos `gesturestart`, `gestureend`, `gesturehold` com payload (nome, confiança, timestamp)	M
RF-ACT-03	Mapear gesto → ação: tecla/atalho (dentro da página), clique em elemento, fala (Web Speech API), navegação, webhook HTTP, evento WebSocket	M
RF-ACT-04	Configurar por gesto: limiar, tempo de sustentação, cooldown, modo (único/contínuo)	M
RF-ACT-05	Botão/atalho de emergência para pausar todo reconhecimento	M
RF-ACT-06	Registro (log) visível dos últimos disparos para depuração	S
RF-ACT-07	Controle de cursor por movimento de cabeça com sensibilidade ajustável e *dwell click*	C
RF-ACT-08	Modo varredura (scanning) com teclado virtual acionado por um único gesto	C

4.4 Perfis e persistência (PRF)

ID	Requisito	Prior.
RF-PRF-01	Salvar perfis (gestos, exemplos, modelo, mapeamentos) em IndexedDB	M
RF-PRF-02	Exportar/importar perfil como arquivo único (`.gestolivre.json`)	M
RF-PRF-03	Vários perfis por dispositivo (ex.: "Maria — cabeça", "Maria — mão direita")	S
RF-PRF-04	Excluir todos os dados com um clique, com confirmação	M
RF-PRF-05	Versionar o esquema do perfil e migrar perfis antigos	S

4.5 Biblioteca (API)

ID	Requisito	Prior.
RF-API-01	Publicar `@gestolivre/core` no npm com tipos TypeScript, ESM e CJS	M
RF-API-02	API declarativa e orientada a eventos (ver exemplo na seção 6)	M
RF-API-03	Funcionar sem a UI: ser possível treinar e inferir só por código	M
RF-API-04	Interface `Classifier` e `FeatureExtractor` extensíveis por plugins	S
RF-API-05	Documentação com guia rápido, referência e exemplos executáveis	M

5. Requisitos não funcionais

ID	Categoria	Requisito
RNF-01	Desempenho	Inferência ponta a ponta (quadro → evento) ⩽ 100 ms em notebook comum sem GPU dedicada
RNF-02	Desempenho	Pelo menos 24 quadros/s com um detector ativo; 15 quadros/s com os três ativos
RNF-03	Desempenho	Treino de até 10 gestos × 50 exemplos em ⩽ 5 s com k-NN e ⩽ 30 s com MLP
RNF-04	Privacidade	Nenhum quadro de vídeo é gravado, enviado ou persistido; apenas landmarks numéricos
RNF-05	Privacidade	Funcionamento offline após o primeiro carregamento (PWA e modelos em cache)
RNF-06	Segurança	HTTPS obrigatório (exigência do `getUserMedia`), Content Security Policy restritiva, sem dependências remotas em tempo de execução além dos modelos fixados
RNF-07	Acessibilidade	A própria interface atende WCAG 2.2 AA: navegação por teclado, leitor de tela, contraste, alvos de toque ⩾ 44 px

ID	Categoria	Requisito
RNF-08	Acessibilidade	Todo tempo e limiar ajustável (para tremor, fadiga, espasmos); sem exigir resposta rápida do usuário
RNF-09	Compatibilidade	Chrome/Edge/Firefox/Safari recentes em desktop; Chrome/Safari em Android/iOS (modo reduzido)
RNF-10	Portabilidade	Biblioteca sem dependência de framework; componentes React como camada opcional
RNF-11	Manutenibilidade	Cobertura de testes ≥ 80% no núcleo; tipagem estrita (`strict: true`)
RNF-12	Observabilidade local	Painel opcional de métricas (FPS, latência, confiança) sem telemetria externa
RNF-13	Licenciamento	Licença permissiva (MIT ou Apache-2.0); respeitar as licenças dos modelos do MediaPipe
RNF-14	Internacionalização	Interface em português do Brasil e inglês desde o início

6. Arquitetura e tecnologias

6.1 Estrutura do repositório

```
gestolivre/
├── packages/
│   ├── core/        # câmera, extração, treino, inferência, eventos
│   ├── ui/          # componentes de gravação, teste e mapeamento
│   ├── actions/     # executores (teclado, clique, fala, webhook)
│   └── bridge/      # (fase 2) app desktop que injeta teclado/mouse
├── apps/
│   └── studio/      # PWA: tela de treino e configuração
├── examples/        # exemplos mínimos (vanilla, React, jogo, slides)
├── spec/            # requisitos, critérios de aceite, rastreabilidade
└── docs/            # guias, referência da API, acessibilidade
```

6.2 Stack

Camada	Tecnologia	Motivo
Linguagem	TypeScript 5 (estrito)	Tipagem e DX para quem consome a biblioteca

Camada	Tecnologia	Motivo
Captura	`getUserMedia`, `MediaStreamTrackProcessor` / `requestVideoFrameCallback`	Quadros com baixa latência
Detecção	**MediaPipe Tasks Vision** (`HandLandmarker`, `FaceLandmarker` com blendshapes, `PoseLandmarker`), backend WASM/GPU	Modelos prontos, rápidos e executáveis no navegador
Aprendizado	**TensorFlow.js** (WebGL/WebGPU/WASM) para MLP e sequência; k-NN próprio em TypeScript puro	Treino local; k-NN sem dependências pesadas
Concorrência	Web Workers, OffscreenCanvas	UI fluida
Armazenamento	IndexedDB (via `idb`)	Perfis e modelos locais
Build	Vite + tsup, monorepo com pnpm workspaces e Changesets	Publicação e versionamento
UI do Studio	React 18, Tailwind CSS, Radix UI (acessível), Zustand	Componentes acessíveis e estado simples
PWA	Workbox / service worker	Uso offline
Testes	Vitest, Testing Library, Playwright (com câmera simulada), jest-axe/axe-core	Unidade, integração, e2e e acessibilidade
Qualidade	ESLint, Prettier, TypeDoc, GitHub Actions	CI e documentação
Bridge (fase 2)	Tauri (Rust) ou Electron + WebSocket local, bibliotecas de injeção de entrada do SO	Controlar aplicativos fora do navegador

6.3 Limitações importantes do navegador (e como tratar)

- **O navegador não pode controlar outros programas.** Teclas e cliques só afetam a própria página. Para controlar o sistema inteiro, é necessário o **Bridge** (app desktop) ou uma extensão de navegador.
- **Abas em segundo plano são limitadas:** o navegador reduz o processamento de abas ocultas. Mitigação: modo Picture-in-Picture, janela destacada ou o Bridge desktop executando a captura.
- **Câmera exige HTTPS ou localhost.**

- **Variação de hardware:** webcams baratas têm baixa taxa de quadros; o sistema deve medir e avisar.

6.4 Exemplo de uso da biblioteca (proposta de API)

```
import { GestoLivre } from "@gestolivre/core";

const g = await GestoLivre.create({
  sources: ["face", "hand"],
  classifier: "knn",
});

await g.camera.start({ mirror: true });

// Treino: grava 40 exemplos do gesto "piscar-esquerdo"
await g.train.record("piscar-esquerdo", { samples: 40, countdown: 3 });
await g.train.record("neutro", { samples: 60, countdown: 3 });
const report = await g.train.fit();   // { accuracy, confusion, perGesture }

// Reação ao gesto
g.on("gesturestart", ({ name, confidence }) => {
  if (name === "piscar-esquerdo") document.querySelector("#proximo").click();
});

g.configure("piscar-esquerdo", { threshold: 0.9, holdMs: 400, cooldownMs:
1000 });

const profile = await g.profile.export(); // salvar/compartilhar
```

7. Processo de desenvolvimento

1. **Spec-Driven Development:** cada requisito deste documento vira um arquivo em `spec/` com ID, critérios de aceite em Dado/Quando/Então e vínculo com testes. Um script em CI falha se existir requisito *Must* sem teste.
2. **Incremental por fatias verticais:** cada iteração entrega algo utilizável de ponta a ponta (ex.: primeira fatia = abrir a webcam, gravar 1 gesto, detectar, mostrar na tela).
3. **Design centrado no usuário com tecnologia assistiva:** testes de usabilidade com pelo menos 3 pessoas com limitações motoras ou com terapeuta ocupacional, desde o protótipo da Fase 1.
4. **Experimentos reprodutíveis:** conjunto de dados próprio, com consentimento, de landmarks (sem imagens), versionado, usado para avaliar classificadores e regressões.

5. **Ciclo:** iterações de 1 a 2 semanas, revisão de PR obrigatória, commits convencionais e Changesets para versionamento semântico.
6. **Definição de pronto:** testes passando, axe sem violações graves, documentação atualizada, matriz de rastreabilidade atualizada.

7.1 Roadmap sugerido

Etapa	Entrega	Duração estimada
0	Repositório, CI, protótipo "webcam + landmarks na tela"	1 semana
1	Núcleo: extração, normalização, k-NN, eventos	2 semanas
2	Studio MVP: gravar, treinar, testar ao vivo, mapear ações simples	3 semanas
3	Perfis, exportar/importar, anti-disparo acidental, PWA offline	2 semanas
4	Acessibilidade da interface, testes com usuários, ajustes	2 semanas
5	Publicação npm, documentação, exemplos, v0.1	1 semana
6	MLP, gestos dinâmicos (janela temporal/DTW)	3 semanas
7	Bridge desktop, varredura, cursor por cabeça	4 semanas

8. Critérios de aceite

8.1 Funcionais (Dado / Quando / Então)

CA-CAM-01 — Permissão e privacidade

- Dado que o usuário abre o Studio pela primeira vez,
- Quando clica em "Ativar câmera",
- Então vê um texto explicando que nenhum vídeo é gravado ou enviado, o navegador solicita permissão, e um indicador "câmera ativa" aparece enquanto ela estiver em uso.

CA-TRN-01 — Treino em menos de dois minutos

- Dado um usuário novo sem perfil,
- Quando ele cria o gesto "joinha", grava 40 exemplos, grava o neutro e clica em Treinar,

- Então o modelo é treinado em ⩽ 5 s e o teste ao vivo mostra "joinha" ao refazer o gesto; todo o fluxo é concluído em ⩽ 2 minutos.

CA-TRN-02 — Qualidade mínima

- Dado um perfil com 3 gestos distintos e a classe neutro, treinados com 40 exemplos cada,
- Quando é executada a validação em blocos temporais,
- Então a acurácia é ⩾ 90% e nenhum gesto tem revocação < 80%.

CA-TRN-03 — Gestos parecidos

- Dado dois gestos cujos exemplos se sobrepõem acima do limiar,
- Quando o usuário treina,
- Então o Studio exibe aviso identificando o par e sugere regravar ou unir os gestos.

CA-ACT-01 — Disparo com sustentação

- Dado o gesto "piscar-esquerdo" com holdMs = 400,
- Quando o gesto dura 250 ms,
- Então nenhuma ação é executada; quando dura ⩾ 400 ms, a ação é executada uma única vez.

CA-ACT-02 — Anti-disparo acidental

- Dado um perfil treinado, com o usuário digitando e conversando por 10 minutos,
- Quando o reconhecimento está ativo,
- Então o número de disparos indevidos é ⩽ 1 por hora com a configuração padrão.

CA-ACT-03 — Pausa de emergência

- Dado o reconhecimento ativo,
- Quando o usuário aciona o botão ou atalho de pausa,
- Então nenhum evento é emitido em ⩽ 200 ms e a interface mostra o estado "pausado" de forma visível.

CA-ACT-04 — Mapeamento de ação

- Dado o gesto "mão aberta" mapeado para "próximo slide" no exemplo de apresentação,
- Quando o usuário faz o gesto com o modelo ativo,
- Então o slide avança uma vez e o log registra gesto, confiança e horário.

CA-PRF-01 — Exportar e importar

- Dado um perfil treinado,
- Quando é exportado e importado em outro navegador/dispositivo,

- Então os gestos e mapeamentos são restaurados e o modelo é reajustado automaticamente a partir dos exemplos (ou carregado) com resultados equivalentes.

CA-PRF-02 — Apagar dados

- Dado que o usuário clica em "Apagar tudo" e confirma,
- Quando a operação termina,
- Então IndexedDB, caches de perfil e modelos locais estão vazios, verificável por teste automatizado.

CA-API-01 — Uso só com código

- Dado um projeto vazio com `@gestolivre/core` instalado,
- Quando o desenvolvedor segue o guia rápido (⩽ 20 linhas),
- Então consegue treinar um gesto e receber o evento `gesturestart` sem usar a UI.

8.2 Não funcionais

ID	Critério de aceite
CA-RNF-01	Em notebook de referência (4 núcleos, sem GPU dedicada), latência mediana ⩽ 100 ms e p95 ⩽ 160 ms com um detector ativo
CA-RNF-02	Teste automatizado confirma que nenhuma requisição de rede transmite quadros, landmarks ou exemplos (inspeção de tráfego no Playwright)
CA-RNF-03	Após o primeiro carregamento, o Studio funciona com a rede desativada
CA-RNF-04	axe-core sem violações graves/críticas; fluxo completo de treino utilizável só com teclado e testado com leitor de tela
CA-RNF-05	Cobertura de testes ⩾ 80% em `core`; tipos estritos sem `any` implícito
CA-RNF-06	Tamanho do pacote `core` ⩽ 150 KB gzip (modelos carregados sob demanda)

8.3 Aceite com usuários reais

ID	Critério
CA-USR-01	Pelo menos 3 participantes com limitação motora completam o fluxo de treino de um gesto próprio sem ajuda técnica, em ⩽ 10 minutos
CA-USR-02	Pelo menos 2 participantes conseguem executar uma tarefa real (ex.: navegar em slides, acionar frase por voz) com taxa de sucesso ⩾ 85% em 20 tentativas
CA-USR-03	Pontuação SUS (System Usability Scale) média ⩾ 75
CA-USR-04	Participantes conseguem ajustar tempo de sustentação e limiar por conta própria ou com instruções claras

9. Riscos e mitigação

Risco	Impacto	Mitigação
Disparos acidentais	Frustração, ações indesejadas	Classe neutro, histerese, sustentação, cooldown, pausa de emergência
Variação de iluminação/câmera	Queda de acurácia	Landmarks normalizados, aviso de qualidade, aumento de dados
Fadiga ou tremor do usuário	Gestos inconsistentes	Tempos configuráveis, gestos de baixo esforço, regravação fácil, modo varredura
Limite do navegador em segundo plano	Reconhecimento pausa	PiP, janela destacada, Bridge desktop
Dados biométricos (LGPD)	Risco legal	Sem imagens, processamento local, exclusão com um clique, política de privacidade clara
Dependência dos modelos do MediaPipe	Mudança de API/licença	Versões fixadas, camada de abstração `FeatureExtractor`
Falsa expectativa de dispositivo médico	Uso inadequado	Aviso claro de que é ferramenta auxiliar, não produto médico

10. Privacidade e ética

- Dados faciais e corporais podem ser considerados dados pessoais sensíveis (biométricos) na LGPD; por isso a decisão de projeto é **não armazenar imagens** e manter tudo local.
- Consentimento explícito e granular para a câmera e para qualquer exportação de perfil.
- Perfis exportados contêm landmarks numéricos que, em tese, caracterizam uma pessoa: tratar como dado pessoal ao compartilhar e avisar o usuário.
- Conjuntos de dados de pesquisa só com consentimento informado e anonimização.
- Diversidade: testar com diferentes tons de pele, idades, tipos de rosto, mobilidade e acessórios (óculos, hijab, barba), para evitar viés de detecção.

11. Ideias de aplicações de exemplo (para a pasta `examples/`)

1. **Passador de slides por gesto** (mão aberta = próximo, punho = anterior).
2. **Comunicador por piscadas:** piscar = escolher, sobrancelha = avançar, voz sintetizada fala a frase.
3. **Teclado de varredura** com um único movimento de cabeça.
4. **Jogo simples** (ex.: dinossauro do Chrome) controlado por boca aberta.
5. **Controle de mídia:** pausar vídeo ao se afastar da tela ou fazer gesto de "pare".
6. **Automação residencial:** gesto aciona webhook (luz, cortina) via Home Assistant.

12. Pontos em aberto

- Escolha do nome e da licença final (MIT vs Apache-2.0).
- Foco inicial: acessibilidade (rosto/cabeça) ou criatividade (mão)? Recomendação: rosto e cabeça primeiro, pois blendshapes entregam sinais prontos e precisos.
- Necessidade do Bridge desktop já no lançamento, ou apenas após validar o Studio.
- Parceria com clínica/associação para recrutar participantes dos testes de usabilidade.

Observação: números de desempenho, limiares e metas de qualidade são pontos de partida e devem ser calibrados com medições reais no hardware de referência.